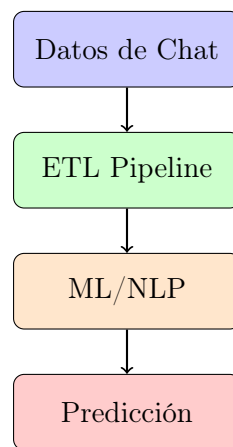


EM-Predictor

Sistema de Predicción de Brotes de Esclerosis Múltiple

Mediante Análisis Lingüístico y Machine Learning



Documentación Técnica v0.4.0

Febrero 2026

Contents

1	Resumen Ejecutivo	2
1.1	Objetivo del Proyecto	2
1.2	Métricas Objetivo y Resultados	2
2	Arquitectura del Sistema	2
2.1	Visión General	2
2.2	Stack Tecnológico	3
3	Pipeline de Datos (ETL)	3
3.1	Extracción de Datos	3
3.2	Features Extraídos	3
3.2.1	Features Lingüísticos (por mensaje)	3
3.2.2	Features Temporales (por día/ventana)	3
3.2.3	Features de Ingeniería	4
3.3	Generación de Labels	4
4	Modelos de Machine Learning	4
4.1	Modelos Baseline	4
4.1.1	Random Forest	4
4.1.2	Gradient Boosting Machine (GBM)	4
4.2	Ensemble Methods	5
4.3	Validación Temporal	5
4.3.1	Walk-Forward Validation con Embargo	5
5	Análisis de Data Leakage	5
5.1	Problema Identificado	5
5.2	Fuentes de Leakage	5
5.3	Correcciones Implementadas	6
6	NLP y Embeddings de Texto	6
6.1	Estado Actual	6
6.2	Propuesta: Sentence Embeddings	6
6.2.1	Modelos Disponibles	6
6.2.2	Implementación Propuesta	6
6.2.3	Features Derivados de Embeddings	7
6.3	Propuesta: Fine-tuning con Datos del Paciente	7
6.4	Propuesta: Análisis de Sentiment con Transformers	8
7	Propuestas de Mejora	8
7.1	Corto Plazo (1-2 semanas)	8
7.2	Medio Plazo (3-4 semanas)	8
7.3	Largo Plazo	9
8	Pipeline Propuesto con NLP	9
9	Conclusiones	9
9.1	Estado Actual	9
9.2	Limitaciones	10
9.3	Pasos Siguietes	10
A	Estructura de Archivos del Proyecto	10

B Comandos de Ejecución

10

1 Resumen Ejecutivo

1.1 Objetivo del Proyecto

El proyecto **EM-Predictor** tiene como objetivo desarrollar un sistema de predicción de brotes de Esclerosis Múltiple (EM) con **7-30 días de antelación**, utilizando:

- Análisis lingüístico de comunicaciones (WhatsApp, Telegram)
- Patrones de actividad temporal
- Features clínicos y de comportamiento
- Modelos de Machine Learning y NLP

1.2 Métricas Objetivo y Resultados

Table 1: Métricas de Rendimiento del Sistema

Métrica	Objetivo	Holdout	Walk-Forward
AUROC (14 días)	> 0.65	0.6932	0.41 ± 0.15
AUPRC	> 0.30	0.3609	–
Brier Score	< 0.25	TBD	–

Hallazgo Crítico: Existe una discrepancia significativa entre métricas de holdout (0.69) y walk-forward validation (0.41), indicando problemas de generalización temporal que han sido identificados y parcialmente corregidos.

2 Arquitectura del Sistema

2.1 Visión General

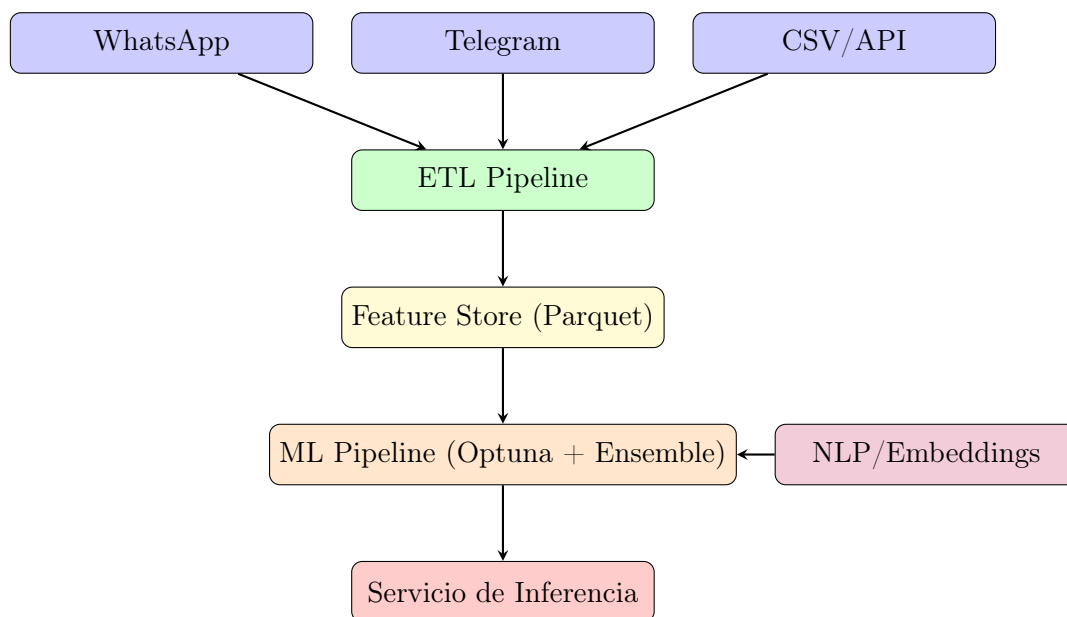


Figure 1: Arquitectura de Alto Nivel del Sistema

2.2 Stack Tecnológico

Table 2: Tecnologías Utilizadas

Componente	Tecnología	Justificación
Training	scikit-learn	Baselines rápidos, bien probados
Temporal	pytorch-forecasting	TFT para series temporales
HPO	Optuna	Pruning eficiente
Embeddings	sentence-transformers	Multilingüe, local, gratuito
Storage	Parquet	Columnar, compresión, fast I/O
DB	TimescaleDB	Time-series optimized
API	FastAPI	Async, autodocs
MLOps	MLflow	Tracking experimentos

3 Pipeline de Datos (ETL)

3.1 Extracción de Datos

El sistema procesa datos de comunicación de pacientes desde múltiples fuentes:

```

1 FORMATOS_SOPORTADOS = ["whatsapp", "telegram", "csv", "jsonl"]
2
3 # Ejemplo de mensaje normalizado
4 mensaje = {
5     "timestamp": "2024-12-01 14:30:00",
6     "sender": "paciente",
7     "text": "Hoy me siento muy cansada...",
8     "metadata": {"source": "whatsapp"}
9 }
```

Listing 1: Formatos de entrada soportados

3.2 Features Extraídos

3.2.1 Features Lingüísticos (por mensaje)

- Longitud: char_count, word_count, sentence_count
- Diversidad léxica: unique_words, type_token_ratio
- Estilo: exclamation_count, question_count, caps_ratio
- Expresividad: emoji_count
- Sentiment: Proxy léxico básico (-1 a +1)

3.2.2 Features Temporales (por día/ventana)

- Volumen: messages_total, messages_mean, messages_std
- Cobertura: num_active_days, coverage
- Actividad: active_hours, night_ratio
- Tendencias: messages_trend, sentiment_trend

3.2.3 Features de Ingeniería

```

1 # Lag features (valores de d as anteriores)
2 df[f"{col}_lag{lag}"] = df[col].shift(lag)
3
4 # Rolling features CON SHIFT para evitar data leakage
5 # IMPORTANTE: shift(1) asegura que solo usamos datos hasta t-1
6 shifted = df[col].shift(1)
7 df[f"{col}_roll{window}_mean"] = shifted.rolling(window).mean()
8 df[f"{col}_roll{window}_std"] = shifted.rolling(window).std()
9 df[f"{col}_roll{window}_trend"] = shifted.rolling(window).apply(
10     lambda x: np.polyfit(range(len(x)), x, 1)[0]
11 )

```

Listing 2: Feature Engineering con prevención de leakage

3.3 Generación de Labels

$$y_{t,h} = \begin{cases} 1 & \text{si existe evento en } [t+1, t+h] \\ 0 & \text{en caso contrario} \end{cases} \quad (1)$$

Donde $h \in \{7, 14, 30\}$ días son los horizontes de predicción.

4 Modelos de Machine Learning

4.1 Modelos Baseline

4.1.1 Random Forest

Configuración optimizada mediante Optuna:

```

1 RandomForestClassifier(
2     n_estimators=58,
3     max_depth=13,
4     min_samples_split=8,
5     min_samples_leaf=4,
6     class_weight="balanced"
7 )

```

4.1.2 Gradient Boosting Machine (GBM)

```

1 GradientBoostingClassifier(
2     n_estimators=89,
3     max_depth=6,
4     learning_rate=0.018,
5     min_samples_split=7,
6     subsample=0.92
7 )

```

4.2 Ensemble Methods

Table 3: Comparación de Métodos Ensemble

Método	AUROC	AUPRC
Random Forest	0.5917	0.3032
GBM	0.6932	0.3609
Logistic Regression	0.1623	0.1635
Voting (Soft)	0.5928	0.3044
Stacking	0.4578	0.2416
Manual Average	0.6511	0.3282

4.3 Validación Temporal

4.3.1 Walk-Forward Validation con Embargo

Para evaluar correctamente modelos de series temporales, implementamos validación walk-forward con:

- **Gap (embargo)**: Período entre fin de train y inicio de test
- **Purge**: Eliminación de últimos N días del train (sus labels miran al futuro)

Algorithm 1 Walk-Forward Validation con Embargo

Require: Dataset D , train_window, test_window, gap, purge

```

1: Ordenar  $D$  por fecha
2:  $t \leftarrow 0$ 
3: while  $t + \text{train\_window} + \text{gap} + \text{test\_window} \leq |D|$  do
4:    $\text{train\_end} \leftarrow t + \text{train\_window} - \text{purge}$ 
5:    $\text{test\_start} \leftarrow t + \text{train\_window} + \text{gap}$ 
6:    $\text{test\_end} \leftarrow \text{test\_start} + \text{test\_window}$ 
7:    $D_{\text{train}} \leftarrow D[t : \text{train\_end}]$ 
8:    $D_{\text{test}} \leftarrow D[\text{test\_start} : \text{test\_end}]$ 
9:   Entrenar modelo en  $D_{\text{train}}$ 
10:  Evaluar en  $D_{\text{test}}$ 
11:   $t \leftarrow t + \text{step}$ 
12: end while
```

5 Análisis de Data Leakage

5.1 Problema Identificado

Se detectó una discrepancia significativa entre métricas:

$$\text{AUROC}_{\text{holdout}} - \text{AUROC}_{\text{walk-forward}} = 0.69 - 0.41 = 0.28 \quad (2)$$

5.2 Fuentes de Leakage

1. **Rolling Features sin Shift**: Los features de ventana móvil incluían datos del día actual ($t = 0$).

2. **Validación sin Gap:** El test comenzaba inmediatamente después del train, pero el target `relapse_in_14d` mira 14 días hacia adelante.
3. **Labels Contaminados:** Los últimos 14 días del train tienen labels que dependen del período de test.

5.3 Correcciones Implementadas

Table 4: Correcciones de Data Leakage

Corrección	Implementación
Shift en rolling	<code>df[col].shift(1).rolling(window).mean()</code>
Gap temporal	Parámetro <code>-gap</code> 14 en walk-forward
Purge de train	Parámetro <code>-purge</code> 14 para eliminar últimos días
Auditoría	Script <code>validate_no_leakage.py</code>

6 NLP y Embeddings de Texto

6.1 Estado Actual

Actualmente el sistema usa un **proxy de sentiment** básico basado en palabras clave:

```

1 def _basic_sentiment(self, text: str) -> float:
2     positive = ["bien", "genial", "feliz", "mejor", "bueno"]
3     negative = ["mal", "fatal", "triste", "peor", "cansad"]
4
5     text_lower = text.lower()
6     pos_count = sum(1 for w in positive if w in text_lower)
7     neg_count = sum(1 for w in negative if w in text_lower)
8
9     return (pos_count - neg_count) / (pos_count + neg_count + 1e-6)

```

Listing 3: Sentiment básico actual

6.2 Propuesta: Sentence Embeddings

6.2.1 Modelos Disponibles

Table 5: Presets de Embeddings

Preset	Modelo	Dimensiones	Velocidad
fast	distiluse-base-multilingual	512	Alta
balanced	paraphrase-multilingual-mpnet	768	Media
quality	multilingual-e5-large	1024	Baja

6.2.2 Implementación Propuesta

```

1 from sentence_transformers import SentenceTransformer
2
3 class TextEmbedder:
4     def __init__(self, model_name="paraphrase-multilingual-mpnet-base-v2"):
5         self.model = SentenceTransformer(model_name)
6
7     def embed_window(self, messages: List[str]) -> np.ndarray:

```



```

8      """Genera embedding agregado para una ventana de mensajes."""
9      if not messages:
10         return np.zeros(self.model.get_sentence_embedding_dimension())
11
12     # Embeddings individuales
13     embeddings = self.model.encode(messages)
14
15     # Agregación: mean pooling
16     return np.mean(embeddings, axis=0)
17
18     def embed_with_temporal_context(self, messages: List[str],
19                                     timestamps: List[datetime]) -> Dict:
20         """Embeddings con contexto temporal."""
21         embeddings = self.model.encode(messages)
22
23         return {
24             "embedding_mean": np.mean(embeddings, axis=0),
25             "embedding_std": np.std(embeddings, axis=0),
26             "embedding_first": embeddings[0], # Inicio de ventana
27             "embedding_last": embeddings[-1], # Fin de ventana
28             "embedding_trend": embeddings[-1] - embeddings[0], # Cambio
29         }

```

Listing 4: Generación de embeddings por ventana temporal

6.2.3 Features Derivados de Embeddings

- **Embedding medio:** Representación semántica general de la ventana
- **Varianza de embeddings:** Diversidad temática
- **Similitud coseno:** Entre ventanas consecutivas (estabilidad)
- **Distancia al centroide "normal":** Detección de anomalías
- **Componentes PCA:** Reducción dimensional para interpretabilidad

6.3 Propuesta: Fine-tuning con Datos del Paciente

```

1 from sentence_transformers import SentenceTransformer, losses, InputExample
2
3 def finetune_patient_model(messages_df, patient_id, epochs=3):
4     """
5     Fine-tune el modelo de embeddings con datos del paciente.
6
7     Usa contrastive learning: mensajes cercanos en tiempo
8     deben tener embeddings similares.
9     """
10    model = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")
11
12    # Crear pares de entrenamiento
13    train_examples = []
14    for i in range(len(messages_df) - 1):
15        # Mensajes del mismo día = similares
16        if messages_df.iloc[i]['date'] == messages_df.iloc[i+1]['date']:
17            train_examples.append(InputExample(
18                texts=[messages_df.iloc[i]['text'],
19                     messages_df.iloc[i+1]['text']],
20                label=0.9 # Alta similitud
21            ))
22
23    # Entrenar

```

```

24 train_dataloader = DataLoader(train_examples, batch_size=16)
25 train_loss = losses.CosineSimilarityLoss(model)
26 model.fit(train_objectives=[(train_dataloader, train_loss)], epochs=epochs)
27
28 model.save(f"models/embeddings_{patient_id}")
29 return model

```

Listing 5: Fine-tuning de embeddings específicos del paciente

6.4 Propuesta: Análisis de Sentiment con Transformers

```

1 from transformers import pipeline
2
3 class AdvancedSentimentAnalyzer:
4     def __init__(self):
5         # Modelo español de sentiment
6         self.classifier = pipeline(
7             "sentiment-analysis",
8             model="nlptown/bert-base-multilingual-uncased-sentiment"
9         )
10
11     def analyze(self, text: str) -> Dict:
12         result = self.classifier(text[:512])[0] # Truncar a 512 tokens
13
14         # Convertir a escala -1 a +1
15         stars = int(result['label'].split()[0]) # "3 stars" -> 3
16         sentiment = (stars - 3) / 2 # Mapear 1-5 a -1 a +1
17
18         return {
19             "sentiment": sentiment,
20             "confidence": result['score'],
21             "raw_label": result['label']
22         }

```

Listing 6: Sentiment analysis con modelos pre-entrenados

7 Propuestas de Mejora

7.1 Corto Plazo (1-2 semanas)

1. **Integrar embeddings reales:** Usar sentence-transformers en lugar del proxy de sentiment básico.
2. **Reducción dimensional:** Aplicar PCA a los 768 componentes de embeddings para evitar curse of dimensionality.
3. **Feature selection:** Eliminar features con alta correlación ($r > 0.95$) para reducir redundancia.
4. **Calibración de probabilidades:** Usar Platt scaling o isotonic regression para calibrar outputs.

7.2 Medio Plazo (3-4 semanas)

1. **Modelo TFT:** Entrenar Temporal Fusion Transformer que puede captar patrones secuenciales complejos.
2. **Multi-paciente:** Añadir datos de segundo paciente para validación cruzada y transfer learning.

3. **Detección de anomalías:** Implementar isolation forest o autoencoders para detectar patrones inusuales.

7.3 Largo Plazo

1. **MiniLLM local:** Integrar modelo de lenguaje pequeño (Phi-2, TinyLlama) para análisis más profundo.
2. **Multi-modal:** Combinar texto con datos de actividad física (si disponibles).
3. **Federated Learning:** Entrenar sin centralizar datos de pacientes.

8 Pipeline Propuesto con NLP

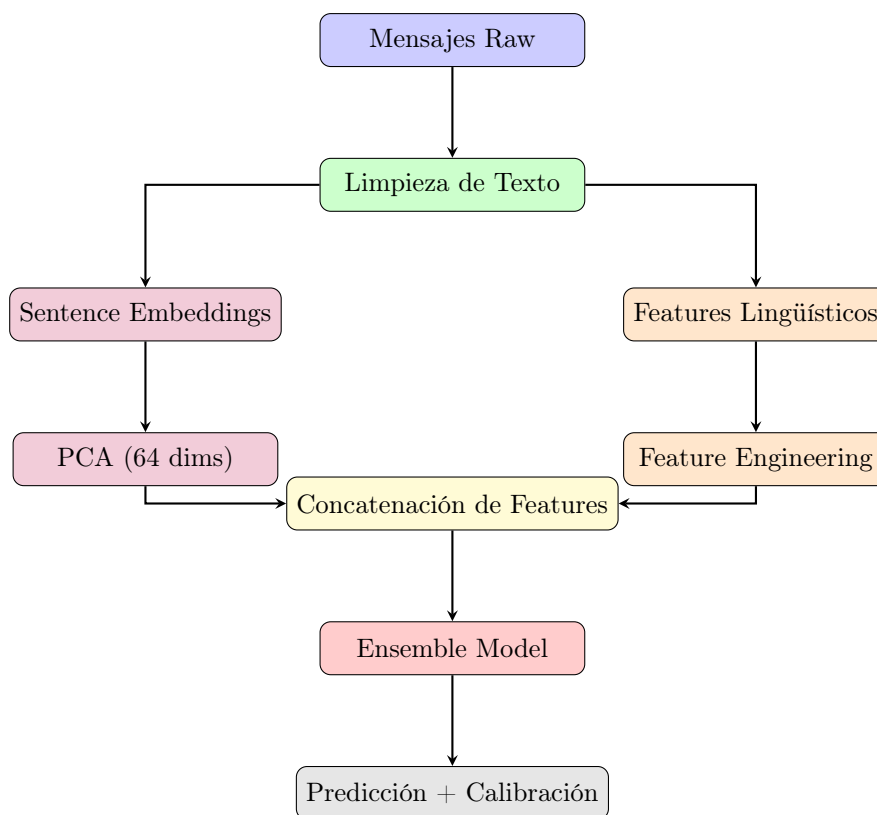


Figure 2: Pipeline Propuesto con Integración de NLP

9 Conclusiones

9.1 Estado Actual

- El sistema ha demostrado **viabilidad técnica** con AUROC 0.69 en holdout.
- Se identificó y corrigió **data leakage temporal** en el pipeline.
- La validación estricta (walk-forward con gap) revela AUROC **0.41**, indicando bajo poder predictivo real.

9.2 Limitaciones

- **Dataset pequeño:** Solo 168 días de un paciente con 5 eventos.
- **Features simples:** El proxy de sentiment actual es muy básico.
- **Falta de validación clínica:** Los labels se basan en detección automática.

9.3 Pasos Siguientes

1. Implementar embeddings de sentence-transformers
2. Añadir datos de segundo paciente
3. Entrenar modelo TFT
4. Validar con equipo clínico

A Estructura de Archivos del Proyecto

```
1 em-predictor/
2   datos/                                # Datos crudos
3       paciente1_whatsapp.txt
4       paciente1_events.csv
5   data/processed/                        # Datos procesados
6       paciente1/
7           daily_features.parquet
8           training_dataset_engineered.parquet
9           ensemble_results.json
10  scripts/
11      etl/                                # Pipeline ETL
12          pipeline.py
13          extract_events.py
14      ml/                                  # Machine Learning
15          feature_engineering.py
16          optuna_simple.py
17          ensemble_model.py
18          walk_forward_validation.py
19          time_series_cv.py
20          validate_no_leakage.py
21  services/                              # Microservicios
22  docs/                                  # Documentación
23  docker-compose.yml
```

Listing 7: Estructura del proyecto

B Comandos de Ejecución

```
1 # ETL
2 python -m scripts.etl.pipeline \
3     --input datos/paciente1_whatsapp.txt \
4     --events datos/paciente1_events.csv \
5     --patient-id paciente1
6
7 # Feature Engineering
8 python scripts/ml/feature_engineering.py \
9     --data-path data/processed/paciente1
10
11 # Optimización de Hiperparámetros
```

```
12 python scripts/ml/optuna_simple.py \  
13     --data-path data/processed/paciente1 \  
14     --n-trials 30  
15  
16 # Validación Walk-Forward con Embargo  
17 python scripts/ml/walk_forward_validation.py \  
18     --data-path data/processed/paciente1 \  
19     --gap 14 \  
20     --purge 14  
21  
22 # Auditoría de Data Leakage  
23 python scripts/ml/validate_no_leakage.py \  
24     --data-path data/processed/paciente1
```

Listing 8: Comandos principales